

Exercice 2
Advanced Computer Architecture

Anton Schelfhout, Yari Vanherpe

16 maart 2024



Instructor: A. Marinšek

1 Introduction

In this lab session we looked at different memory types and coalesced memory in a GPU. We calculated the gray scale of an Image in 2 different ways, once with the RGB-values in the normal order (rgb rgb rgb) and once grouped by color (rrr ggg bbb). We also did some matrix multiplication using different types of memory and looked at the performance difference.

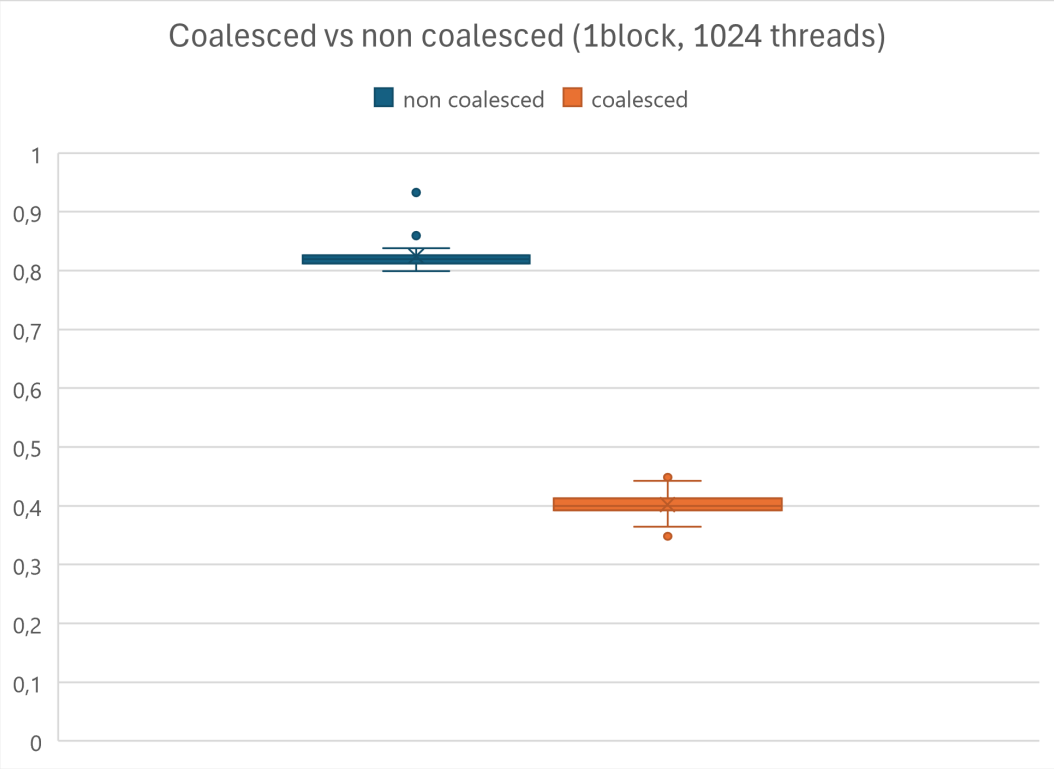
2 Exercise 1: Coalesced memory access

2.1 Method

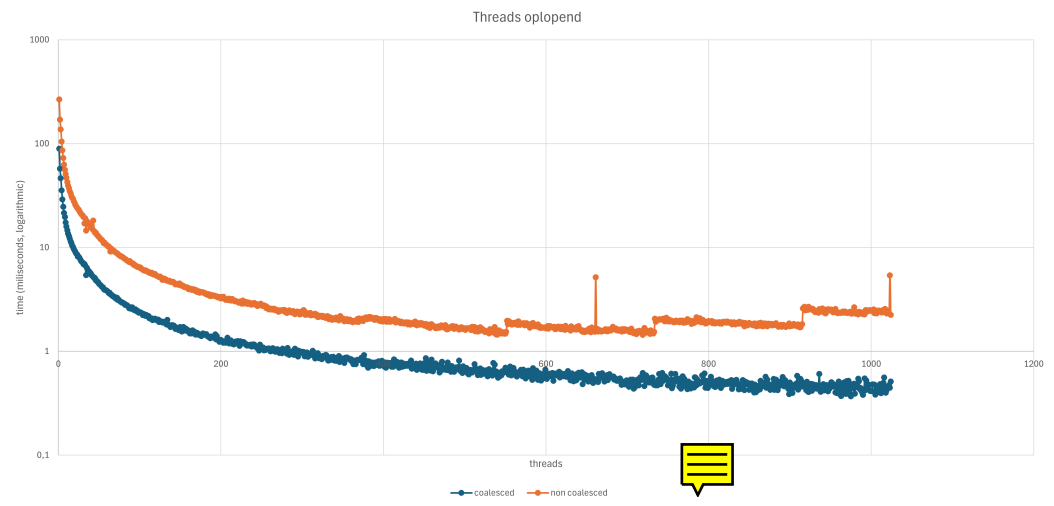
In this exercise we calculated the gray scale of an image. We used two methods, one with coalesced memory access, and one without. For the coalesced method, we first reordered the array of pixels so it would have the form "RRR...GGG...BBBïnstead of its original sequence of "RGBRGBRGB...". Coalesced memory should have an advantage because of the needed memory being grouped together and thus also being loaded together. In The first case there is coalesced memory between threads in 1 warp (all r-values can be read at once) while the second variation has coalescing per thread (each thread can read the r, g and b-value at once).

2.2 Observations

The results of both methods are given in figure1 and 2. Note that the colors in the box plot are reversed compared to the other graph.
When looking at these figures, we conclude that the coalesced approach is fastest. This is in line with what we expected (as is explained in previous section).



Figuur 1: Comparison for 1 block and 1024 threads (50 measurements)



Figuur 2: Speed comparison for multiple threads

3 Exercise 2: Global, shared, and constant memory

3.1 Method

In the second exercise, we looked at the difference between global, shared and constant memory. Global memory is accessible for all threads, regardless of the block it resides in. Global memory is rather far away from the cores. Shared memory is only accessible for threads of the same block, it is lives on the chip and is thus much closer to the cores. The constant memory also lives off-chip, and is accessible for all threads. The difference between global memory is that constant memory is read-only. Shared and constant memory are rather small (48kB and 64kB), while the global memory is larger.
When using the constant and shared memory there is extra overhead because the data has to be copied from the global memory. Because of this, we suspect for small arrays, the overhead will be rather large compared to the performance gained by using this memory (especially for constant memory). For larger inputs, the shared memory should be the fastest (due to its proximity to the GPU cores).

3.2 Observations

When looking at table1 we conclude that the shared memory is the fastest. This makes sense because the memory is much closer to the cores than in the other cases. We also conclude that using constant memory is often faster than only using global memory, but not always. We suspect this depends on the time needed to copy all the values to constant memory, which is sometimes larger than the time gained by using the constant memory.

Also note that the processing times do not become larger as the matrix becomes larger, this is because we calculate the amount of blocks and threads within a block based on the size of the matrix, and the block_size constant (the size of a block which moves over the matrix).

During the lab sessions we encountered a particular behavior in the constant memory on matrices with N = 64. When the block_size became larger than 32 the process time dropped significantly. We suspect this had something to do with caching because when the block size was larger than 32 only one block was used, so only one core. Strangely when trying to reproduce this behavior at home we encountered normal behavior. Maybe its because we went from 10 runs to 50 runs (to calculate the average processing time) and the outliers are not visible as much in the average.

matrix size	**1**	**6**				**64**				**90**
Block Size	**1**	**1**	**2**	**6**	**7**	**16**	**31**	**32**	**33**	**22**
global memory	10.4653	20.1734	10.2598	13.7216	13.4349	13.0246	11.8982	12.0634	12.5747	17.3466
shared memory	6.13376	12.1242	10.3834	13.1891	12.0218	9.46176	10.6502	10.8742	11.4899	11.6109
constant memory	7.6288	16.384	10.8147	18.391	12.9434	10.0966	10.6086	12.1446	14.6835	12.697

Tabel 1: Speed Comparison (in microseconds)